Poznaj oprogramowanie ERP,
które pracuje na Twój sukces
przez 365 dni w roku!

**Systemowy rozwój
Twojej firmy.**

API - Studium przypadku

Spis treści

API - Studium przypadku 3
Kreatory (wizards) 4
Obsługa okna dialogowego wyboru pliku 7
Wykorzystanie zadań z poziomu definicji dokumentów do tworzenia nowych typów dokumentów 8
Kafel (Brick) jako zwykły przycisk 10
Autonumeracja dokumentów na własnym dodatku 11
Wykorzystanie LangContext 12
Tabela rozszerzająca i jej widok 14
Widok niepowiązany z tabelą bazy danych 15
Usuwanie obiektu a cecha referencyjna 16
Własny lookup 19
Zapis zmian w konstrukcji podraportów repx 24
Pieczątki oddziałów w nagłówkach raportów repx 27

API - Studium przypadku

W skład grupy *Studium przypadku* wchodzą wszystkie artykuły zawierające wiedzę z rozwiązywania zagadnień programistycznych typu studium przypadku, obejmuje to wyłącznie zagadnienia ogólno systemowe. W tym również rozwiązanie problemów typu studium przypadku.

Kreatory (wizards)

Obsługa kreatorów (wizardów) do formularza. Kreatory obsługiwane są jako kolejny style Simple/Tab/Tree/Wizard. Wystarczy, że obiektowi przypiszemy odpowiedni atrybut, żeby formularz otworzył się inaczej. Kolejne zakładki formularza stają się kolejnymi krokami wizarda. Na końcu, jeżeli obiekt implementuje `ICommittable`, to metoda `OnCommitting` może zwrócić kolejny obiekt, który zostanie wyświetlony w tym samym oknie jako kolejne kroki.

Przykład:

Definicja dwóch klas dla kolejnych kroków kreatora:

```
[DataFormStyle(UseWizard=true)]
public class Step1 : ICommitable {

	public string Nazwa1 { get; set; }

	public object OnCommitting(Context cx) {
		return new Step2(this);
	}

	object ICommittable.OnCommitted(Context cx) {
		return null;
	}
}

[DataFormStyle(UseFinalWizard=true)]
public class Step2 : ICommitable {

	private readonly Step1 step1;

	public Step2(Step1 step1) {
		this.step1 = step1;
	}

	public string Nazwa2 { get; set; }

	public decimal Kwota { get; set; }

	public object OnCommitting(Context cx) {
		return "Nazwa1: "+step1.Nazwa1+"\nNazwa2: "+Nazwa2+"\nKwota: "+Kwota;
	}

	object ICommittable.OnCommitted(Context cx) {
		return null;
	}
}
```

Dwie klasy `Step1` i `Step2`. Dla `step1` jest `[DataFormStyle]`, który określa czy jest to wizard i czy jest to ostatnia klasa w ciągu. Chodzi o napis na ostatnim przycisku, zamiast **Dalej** będzie **Gotowe**.

Po przejściu przez pages dla `Step1` wołany jest 'OnCommitting', który zwróci obiekt `Step2`. Rozpoczyna się przejście przez zakładki drugiego obiektu. Po ostatniej zakładce naciskamy **Gotowe** i uruchamiana jest druga metoda `OnCommitting`, która wyświetla messagebox z danymi wprowadzonymi w wizardzie.

Dla każdej klasy zdefiniowałem po dwie zakładki:

Step1.Ogole.pageform.xml

```
<DataForm xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns="http://www.enova.pl/schema/form.xsd" >

  <Page DataContext="{DataSource}">
    <Group CaptionHtml="Pierwszy krok">
      <Field EditValue="{Nazwa1}"></Field>
    </Group>
  </Page>

</DataForm>
```

Step1.Ogolne2.pageform.xml

```
<DataForm xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns="http://www.enova.pl/schema/form.xsd" >

  <Page DataContext="{DataSource}">
    <Group CaptionHtml="Pierwszy krok - kontynuacja">
      <Field CaptionHtml="Wartość ta sama, ale na kolejnej zakładce" EditValue="{Nazwa1}"></Field>
    </Group>
  </Page>

</DataForm>
```

Step2.Ogolne.pageform.xml

```
<DataForm xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns="http://www.enova.pl/schema/form.xsd"
Priority="10">

  <Page DataContext="{DataSource}">
    <Group CaptionHtml="Drugi krok">
      <Field EditValue="{Nazwa2}"></Field>
    </Group>
  </Page>

</DataForm>
```

Step2.Ogolne2.pageform.xml

```
<DataForm xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns="http://www.enova.pl/schema/form.xsd"
Priority="20">

  <Page DataContext="{DataSource}">
    <Group CaptionHtml="Drugi krok - jeszcze wpiszmy kwotę">
      <Field EditValue="{Kwota}"></Field>
    </Group>
  </Page>

</DataForm>
```

Monit o plik

Wersja okienkowa została uzbrojona w zdolność obsługi `FileDialogInfo`. Ten obiekt służy do przekazania informacji o tym, jak ustawić okno dialogowe wyboru pliku. Pozwala między innymi na:

- ustawienie tytułu okna,
- ustawienie filtrów plików,
- ustawienie folderu domyślnego.

Rzecz dotyczy tylko i wyłącznie:

```
<Field EditValue="{Plik}" Class="FileEdit SaveFileEdit"></Field>
```

Użycie sprowadza się do takiej prostej implementacji:

```
public string Plik { get; set; }

public object GetListPlik()
{
    return new FileDialogInfo
    {
        Title = "Wybierz plik",
    }
}
```

Klasa **FileDialogInfo** posiada następujące metody służące do filtrowania plików:

`AddXmlFilter()` Dodaje filtr danych dla plików w formacie XML

`AddTxtFilter()` Dodaje filtr danych dla plików tekstowych TXT

`AddAllFilesFilter()` Dodaje filtr danych dla dowolnych plików (wszystkie pliki)

`AddFilter(string desc, string filter)` Dodaje dowolny filtr. Pierwszy parametr to opis znaczenia dodawanego rozszerzenia, a drugi to filtr np. "*.csv"

Dodatkowo obsługę wyboru wielu plików sprowadzono jedynie do odpowiedniej sygnatury właściwości i tak wyświetlenie okna dialogowego dla:

```
public string[] Plik { get; set; }
```

spowoduje włączenie trybu umożliwiającego wybór więcej niż jednego pliku.

Definicje zadań

Konfiguracja systemu dopuszcza dynamiczne definiowanie dowolnych formularzy dla nowo tworzonych dokumentów. Dostęp do tej funkcjonalności istnieje np. z poziomu Opcje -> Kadry i Płace -> Kadry -> Definicje zadań. Zdefiniowane w ten sposób zadania mogą być wykorzystane w obiegu dokumentów w pulpicie pracownika/kierownika (Zadania i wnioski -> Inne wnioski).

Ideą zastosowania dokumentów jest możliwość dynamicznego przydzielania zadania do wskazanego pracownika w zależności od statusu jaki przyjmuje w danym momencie dokument. Zakładka 'Stany' - tutaj użytkownik definiuje w jakim stanie może zostać pozostawiony dokument po edycji przez jednego z pracowników. Można założyć, że stan dokumentu wiąże się z osobą akceptującą dany dokument na wybranym szczeblu (może to być konstrukcja Pracownik -> Kierownik -> Prezes). Musi zatem istnieć stan dokumentu gdy zostaje wprowadzony do systemu oraz stan pośredniej i końcowej akceptacji. Na formularzu stanu realizacji zadania pole 'Aktywny' wpływa na to, czy dokument, gdy przyjmie wskazany stan, będzie jeszcze dopuszczony do dalszej edycji. Ustawienie pola 'Aktywny' na wartość 'NIE' wiązać się powinien z końcową akceptacją (lub brakiem akceptacji) danego dokumentu. Inne stany z wartością pola 'Aktywny' na 'TAK' będą zatem stanami pośrednimi pomiędzy poszczególnymi szczeblami akceptacji.

Zakładka 'Kod kalkulatora zadań' - tutaj znajduje się algorytm przepływu zadania do wskazanych pracowników.

```
public class ZPN_ZadanieKalkulator : ZadanieKalkulatorBase {

public ZPN_ZadanieKalkulator(Zadanie zadanie) : base(zadanie) {
}

public Zadanie Zadanie {
        get { return zadanie; }
}

public void StanZadania_Changed(StanZadania Poprzedni) {
        if (Zadanie.StanZadania.Nazwa == "Wprowadzony") {
                Pracownik pracownik1 = ...; // tutaj wybór pracownika (np. ze struktury
podległościowej)
                Zadanie.Kontrahent = pracownik1;
        }
        else
        if (Poprzedni != Zadanie.StanZadania && (Zadanie.StanZadania.Nazwa == "Zatwierdzony" ||
Zadanie.StanZadania.Nazwa == "Odrzucony")) {
                Pracownik pracownik2 = ...; // tutaj wybór pracownika (np. ze struktury
podległościowej)
                Zadanie.Kontrahent = pracownik2;
        }
}
}
```

Metoda void StanZadania_Changed(StanZadania Poprzedni) pozwala na wskazanie nowej osoby do której ma trafić edytowany dokument. W podanym przykładzie pokazane jest w jaki

sposób kontrolować zmianę stanu wniosku (robi to pracownik z poziomu interface wybierając odpowiedni stan z listy rozwijalnej). Parametrem metody jest właściwość StanZadania (w nazwanej zmiennej 'Poprzedni'). W niej znajduje się stan zadania przed edycją. W zmiennej 'Zadanie.StanZadania' znajduje się aktualny stan zadania po zapisaniu formularza przez pracownika. Porównując te stany możemy w odpowiedni sposób zaprogramować algorytm tak, że zadanie zmieniając stan z jednego na drugi, skieruje to zadania do odpowiedniego pracownika (poprzez wskazanie obiektu pracownika w zmiennej 'Zadanie.Kontrahent'). Obieg dokumentu trwa tak długo jak algorytm będzie wskazywał nowego pracownika.

Zakładka 'Użytkownika NET' - tutaj użytkownik systemu ma możliwość zaprojektowania własnego formularza z dowolną ilością pól związanych z obiektem zadania. Gdy braknie nam jakiegoś pola, które powinno zostać wyświetlone na formularzu - możemy wykorzystać cechy utworzone dla tabeli 'Zadania'. Przeciągając pola z panelu lewego do prawego, wskazujemy jakie pola znajdą się na edytowalnym formularzu (jego podgląd jest dynamicznie renderowany na dole formularza). Dodatkowo, dla każdego pola które znajdzie się na formularzu, możemy zdefiniować własną nazwę, rozmiar oraz dodatkowo wskazać układ w stosunku do pola poprzedniego.

Tak zaprojektowany formularz jego gotowy aby wprowadzić go w obieg pulpitu pracownika/kierownika (nie można zapomnieć o ustawieniu praw danych dla operatorów logujących się w pulpitach + na zakładce 'Opcje', sekcja 'enovaNet' ustawić dostęp 'Edycja' oraz ustawić wartość pola Widoczne w module na 'Kadry').

Praktyczne zastosowanie definiowalnych zadań zostało przedstawione na podstawie formularza 'Praca w nadgodzinach' - opis działania oraz przykładowa definicja została umieszczona w załączniku do artykułu.

Pliki do pobrania

wniosek.xml (23.71 KB)

Opis.docx (317.88 KB)

Kafel (Brick) jako zwykły przycisk

Istnieje możliwość obsługi kafla (Brick) jako zwykłego przycisku. Żeby utworzyć taki kafel należy wykonać poniższe operacje.

Utworzyć klasę implementującą następujący interfejs:

```
public interface IFolder4 : IFolder2 {
     void OnBrickButtonClicked(Soneta.Forms.Bars.BrickMenuItem item);
     void OnBrickButtonDragged(Soneta.Forms.Bars.BrickMenuItem item);
}
```

i zarejestrować jako folder:

```
[assembly: FolderView("Kafel z akcją", FolderType = typeof(MojaKlasa), BrickColor =
FolderViewAttribute.BlueBrick)]
```

Metoda `OnBrickButtonClicked` pozwala wykonać akcję po kliknięciu w kafel.

```
public class SampleFolder : IFolder4 {
    public void OnBrickButtonClicked(BrickMenuItem item) {
        // Akcja po kliknięciu kafla
    }
}
```

Metoda `OnBrickButtonDragged` pozwala wykonać akcję podczas przeciągania kafla (można zareagować na Drag&Drop).

Autonumeracja dokumentów na własnym dodatku

Załączony dodatek (oparty o dodatkowe tabele) prezentuje przykład wykorzystania mechanizmu autonumeracji dokumentów w tworzonych tabelach.

Pliki do pobrania

autonumeracja.zip (1.76 MB)

Wykorzystanie LangContext

Podczas tworzenia oprogramowania wielojęzycznego zdarza się że stajemy przed problemem polegającym na wielorakim użyciu danego słowa w różnych kontekstach a co za tym idzie, jego tłumaczeniu. Prostym przykładem takiej sytuacji może być słowo „uprawnienie" które może być tłumaczenie jako „entitle", „permission" i być używane w różnych kontekstach np. administracyjnych programu lub innych. Rozwiązaniem podobnych dylematów staje się atrybut LangContext, który w prosty sposób umożliwia nam swojego rodzaju identyfikacje, indeksację umiejscowienia danej frazy a następnie przetłumaczenie jej w zależności od kontekstu użycia. Standardowo, w systemie znajduje się kilka podobnych przypadków, w przypadku podejrzenia użycia frazy mającej tłumaczenia różniące się kontekstowo, należy zajrzeć do zakładki tłumaczeń do standardowego słownika systemu, co zostanie pokazane w dalszej części artykułu. Aby skorzystać z możliwości kontekstowego tłumaczenia we własnych dodatkach do systemu, należy zastosować atrybut LangContext odpowiednio dla danego elementy systemu. Poniżej przykład użycia przy właściwości, z użyciem atrybutu caption:

```
[Caption("Uprawnienie", LangContext = "Prawa Test")]
```

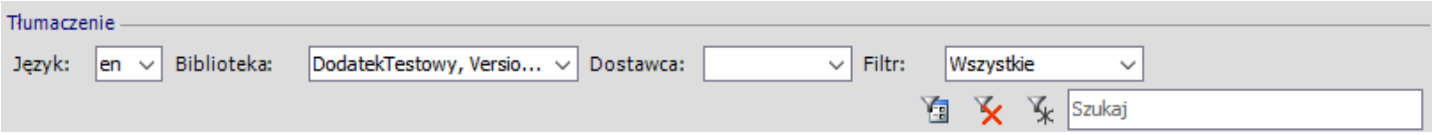
Atrybutu LangContext można również użyć w definicji bazy oraz jej elementów. Poniżej przykłady dodania atrybutu do nazwy bazy danych:

```
<table name="Entitle" tablename="Entitles" guided="Root" config="true"
     caption="Uprawnienie" tablecaption="Uprawnienia" langcontext="Prawa Entitle"
namespace="Soneta.Business.App">
```

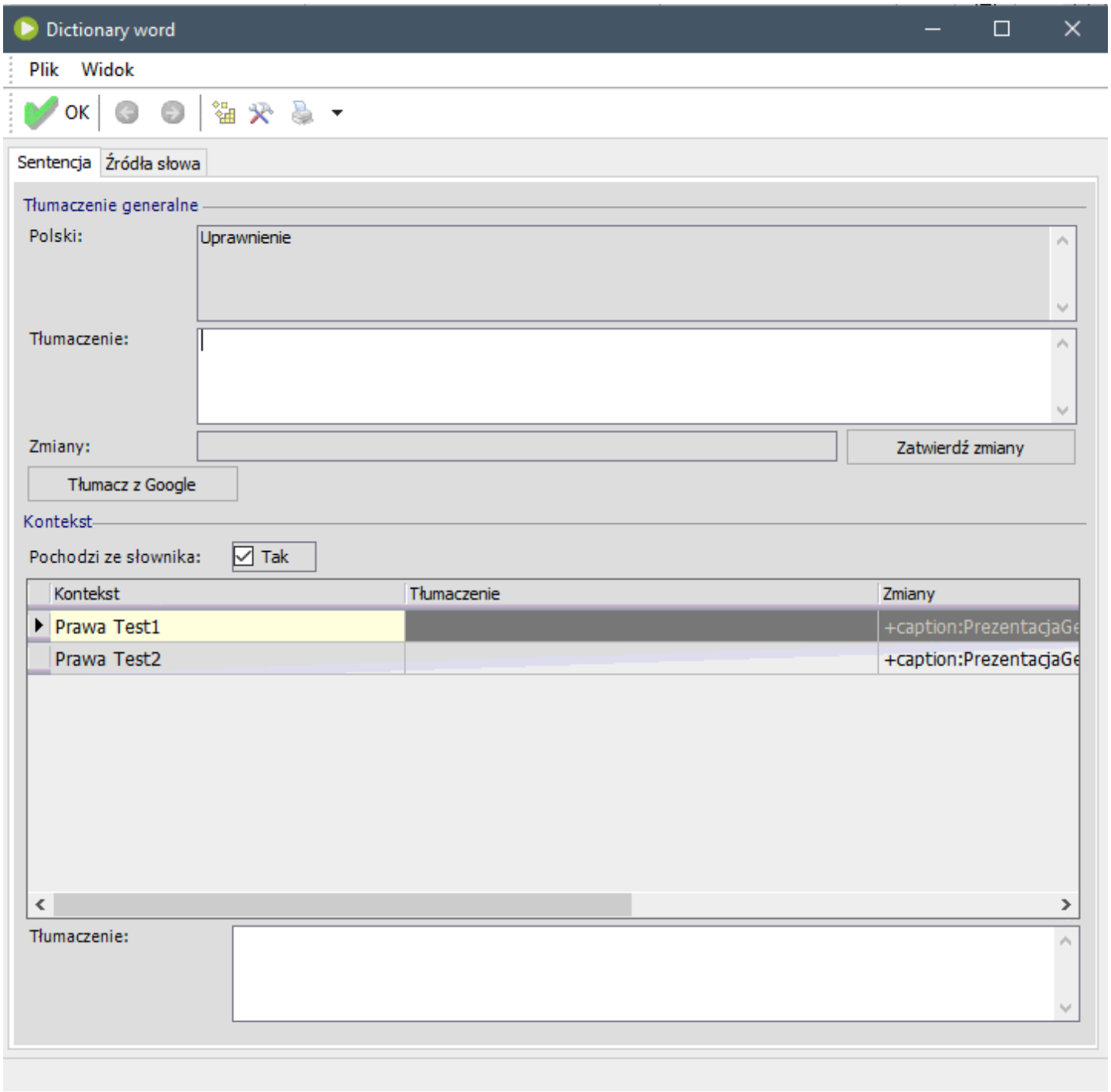
Następny przykład przedstawia użycie atrybutu dla nazwy kolumny bazy danych:

```
<col name="Permission" type="AccessRights" category="Ogólne" caption="Uprawnienie"
langcontext="Prawa Rekord" description="Określa rodzaj uprawnienia"/>
```

Powyższe zmiany w kodzie powodują dodanie do słownika określonego słowa konteksty. W zależności od rodzaju kontekstu, otrzymamy inne tłumaczenie dla tego samego słowa. Po dokonaniu zmian w kodzie, należy wygenerować nowy słownik i uzupełnić tłumaczenia dla nowo dodanych fraz i ich kontekstów. W praktyce należy wejść do menu tłumaczeń dostępnych w *Narzędzia/Opcje/Systemowe/Tłumaczenia* i wygenerować nowy słownik z kodu. Aby wygenerować słownik powyższym menu wybrać żądany język w polu Biblioteka wybrać assembly dodatku.



Następnym krokiem jest wygenerowanie nowego słownika z kodu przy pomocy przycisku Słownik z kodu zlokalizowanego w lewym dolnym rogu widoku. Po zapełnieniu listy tłumaczonymi frazami przechodzimy do fraz, które dodaliśmy a nie są przetłumaczone (w tym celu możemy zastosować filtr). Po odszukaniu frazy, należy wejść w jej definicję i przy odpowiednim kontekście podać tłumaczenie.



Na formularzu znajdują dwa pola do wprowadzenia tłumaczeń. Pierwsze z nich, znajdujący się u góry zawiera tłumaczenie stosowane w sytuacji gdy nie jest zdefiniowany konkretny kontekst dla frazy. Pole znajdujące się na dole widoku służy do wprowadzania tłumaczenia kontekstowego. Ostatnim krokiem, po przetłumaczeniu zapisanie słownika do pliku *.dct. Przy zapisywaniu należy zachować pewną konwencję, przykładowo: ***Soneta.en.dct*** gdzie: ***Soneta*** - własna nazwa słownika, ***en*** – informacja dotycząca języka obsługiwanego przez słownik (CultureInfo.TwoLetterISOLanguageName) ***dct*** – rozszerzenie słownika Zapisany w poprawny sposób słownik należy umieścić w katalogu z dodatkiem.

Dodatek z przykładem tabeli rozszerzającej i jej widoku

Dodatek wnosi tabelę certyfikatów, która jest powiązana z istniejącą tabelą kontrahentów.

Widok znajduję się w menu *CRM/Kontrahenci i certyfikaty (dodatek)*.

Implementacja dodatku prezentuje sposoby rozwiązań zagadnień:

1. Budowa tabeli rozszerzającej czyli takiej, która posiada klucz obcy do tabeli nadrzędnej - **ContractorCertificates.business.xml**
2. Budowa widoku tabeli rozszerzającej prezentującej również dane z tabeli nadrzędnej.
3. Budowa kontrolek filtrów na widoku.
4. Dodanie rekordu do tabeli rozszerzającej (certyfikatu) z domyślną wartością w momencie dodawania nowego kontrahenta.
5. Zabezpieczenie przed usunięciem rekordu z tabeli rozszerzającej. Usunąć rekord można tylko wraz z obiektem nadrzędnym.
6. Budowa okna edycji rekordu z tabeli rozszerzającej.

Uwaga: Po uruchomieniu dodatku widok *Kontrahenci i certyfikaty (dodatek)* będzie pusty. Aby wygenerować rekord certyfikatu należy dodać nowego kontrahenta z poziomu widoku kontrahentów.

Na widoku *Kontrahenci i certyfikaty (dodatek)* pojawi się certyfikat z domyślną nazwą [brak]. Z poziomu widoku można edytować tą właściwość certyfikatu [Nazwa] poprzez dwuklik na wierszu.

Pliki do pobrania

SdkTableExtensionExample.zip (109.38 KB)

Dodatek z przykładem widoku niepowiązanego z tabelą bazy danych

Widok znajduje się w menu *SdkDbFreeView (dodatek)/Pracownicy*.

Implementacja dodatku pokazuje w jaki sposób można zbudować widok z dowolnymi danymi.

Widok wyposażony jest w przyciski dodaj nowy rekord do widoku oraz usuń.

Możliwa jest edycja rekordu poprzez dwuklik na wierszu w kolumnie 'Kod' i uruchomienie okna edycji.

Pozostałe kolumny widoku są edytowane w sposób EditInPlace.

Ten sam widok został podpięty do zakładek w oknie edycji kontrahenta jako przykład tzw extendera.

Pliki do pobrania

SdkDbFreeViewExample.zip (102.7 KB)

Usuwanie obiektu, który jest wartością cechy referencyjnej

Standardowa funkcjonalność usuwania nie pozwoli usunąć obiektu, który jest istniejącą wartością cechy referencyjnej.

Przy próbie operator otrzyma komunikat z informacją że operacji nie można wykonać z powodu istniejącego powiązania z cechą.

Aby usunąć taki obiekt trzeba wpierw ręcznie usunąć jego powiązania z cechami referencyjnymi.

Implementacja poniższego dodatku przedstawia rozwiązanie zagadnienia:

Jak usunąć obiekt a wraz z nim automatycznie usunąć istniejące powiązania z wartościami wskazanej cechy referencyjnej?

Przedstawione są dwa sposoby:

- Funkcja OnDeleting
- Worker

OnDeleting

W klasie Row usuwanego obiektu należy nadpisać wirtualną funkcję z klasy RowBase - OnDeleting()

Funkcja jest wywoływana gdy operator uruchamia funkcjonalność usuwania obiektu.

Transakcja usuwania wartości cech musi być wykonania w oddzielnej sesji.

Taka oddzielna sesja musi zostać zapisana przed sesją, która jest odpowiedzialna za usuwanie danego obiektu.

Próba wykonania transakcji usuwania wartości cech bezpośrednio w sesji odpowiadającej za usuwanie obiektu zostanie zablokowana.

```
public class Mediator : MediatorAddonModule.MediatorRow
{
    protected override void OnDeleting()
    {
        base.OnDeleting();
        ClearMediatorFeatureForDocuments();
    }

    private void ClearMediatorFeatureForDocuments()
    {
        using (var internalSession = Session.Login.CreateSession(false, false, "Internal"))
        {
            using (var transaction = internalSession.Logout(true))
            {
```

```
                HandelModule handelModule = HandelModule.GetInstance(internalSession);

                Key<DokumentHandlowy> documents = handelModule.DokHandlowe.WgKontrahent[new
FieldCondition.Equal("Features.MediatorFeature", this)];

                foreach (var document in documents)
                    document.Features.Remove("MediatorFeature");

                transaction.Commit();
            }
            internalSession.Save();
        }
    }
}
```

Worker

Worker na poziomie widoku, który w osobnej sesji wykonuje usuwanie wartości cech.

Natomiast transakcja usuwania danego obiektu odbywa się już w sesji workera.

```
    private void DeleteMediator()
    {
        using (var internalSession = Session.Login.CreateSession(false, false, "Internal"))
        {
            using (var transaction = internalSession.Logout(true))
            {
                HandelModule handelModule = HandelModule.GetInstance(internalSession);

                Key<DokumentHandlowy> documents = handelModule.DokHandlowe.WgKontrahent[new
FieldCondition.Equal("Features.MediatorFeature", CurrentMediator)];

                foreach (var document in documents)
                    document.Features.Remove("MediatorFeature");

                transaction.Commit();
            }
            internalSession.Save();
        }

        using (var transaction = Session.Logout(true))
        {
            CurrentMediator.Delete();
            transaction.Commit();
        }
    }
```

W załącznikach projekt dodatku Sdk oraz plik xml opisujący cechę referencyjną, która współgra z implementacją dodatku MediatorAddon.

Pliki do pobrania

SdkDeletingReferenceFeature.zip (91.82 KB)

MediatorFeature.xml (1.73 KB)

Tworzenie własnych Lookup-ów

Załączony dodatek PrzykładLookup prezentuje przykład wykorzystania własnych lookupów dodanych do obiektu Kontrahent.

PrzykładLookup składa się z następujących konstrukcji osadzonych w warstwie UI:

- Kontrahent.lookup.pageform.xml
- Kontrahent.lookupform.xml
- Kontrahent2.lookupform.xml
- Klasa KontrahentExtender.cs
- Klasa PrzykładowyLookup.ViewInfo.cs
- Klasa Kontrahenci.ViewInfo.cs

Kontrahent.lookupform.xml to domyślny lookup dla obiektu Kontrahent

```
<!-- domyślny lookup dla obiektu -->
<DataForm Contexts="License.All" xmlns="http://www.enova.pl/schema/form.xsd">
  <Grid Name="List" IsToolbarVisible="false">
    <Field EditValue="{Kod}" />
    <Field EditValue="{Nazwa}" />
    <Field EditValue="{NIP}" />
    <Field EditValue="{Adres}" />
  </Grid>
</DataForm>
```

Kontrahent2.lookupform.xml to dedykowany lookup dla obiektu Kontrahent

```
<!-- dedykowany lookup dla obiektu -->
<DataForm Contexts="License.All" xmlns="http://www.enova.pl/schema/form.xsd">
  <Grid Name="List" IsToolbarVisible="false">
    <Field EditValue="{Kod}" />
    <Field EditValue="{NIP}" Width="40" />
  </Grid>
</DataForm>
```

Kontrahent.lookup.pageform.xml - pageform oparty na Extenderze obiektu

```
<!-- pageform oparty na extenderze obiektu -->
<DataForm xmlns="http://www.enova.pl/schema/form.xsd"
          xsi:schemaLocation="http://www.enova.pl/schema/ http://www.enova.pl/schema/form.xsd"
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
          xmlns:xsd="http://www.w3.org/2001/XMLSchema"
          RightName="Page:LookupyPage"
          Priority="100"
          DataType="Soneta.CRM.Kontrahent,Soneta.CRM">  <-- podpięcie pageforma pod obiekt
Kontrahenta w Soneta.CRM
  <Page Name="LookupyPage" CaptionHtml="Lookupy" DataContext="{new KontrahentExtender}">
    <Group LabelWidth="14">
      <Field EditValue="{Kontrah0}" Width="38"/>
      <Field EditValue="{Kontrah1}" Width="38"/>
      <Field EditValue="{Kontrah2}" Width="38"/>
      <Field EditValue="{Kontrah3}" Width="38"/>
      <Field EditValue="{Kontrah4}" Width="38"/>
    </Group>
  </Page>
</DataForm>
```

Klasa KontrahentExtender.cs klasa zawierająca miedzy innymi metody GetList, które zwracają przefiltrowane obiekty.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Soneta.Business;
using Soneta.Business.UI;
using Soneta.Business.App;
using Soneta.CRM;
using PrzykladLookup.UI;

[assembly: Worker(typeof(KontrahentExtender))]  Rejestracja Extendera jako Worker

namespace PrzykladLookup.UI
{
    class KontrahentExtender
    {
        [Context]
        public Session Session { get; set; }

        [Context]
        public Login Login { get; set; }

        [Context]
        public Kontrahent Kontrahent { get; set; }

        public object GetListKontrah1()
        {
            return new object[] { Session.GetCRM().Kontrahenci.WgKodu["Abc"] };
        }
```

```
        /* lookup tworzony na podstawie SubTable */
        public object GetListKontrah2()
        {
            Kontrahent client;
            string path = nameof(client.Nazwa);
            RowCondition condition = new FieldCondition.Equal(path, "ASPEN GmbH");

            SubTable subTable = Session.GetCRM().Kontrahenci.WgNazwy;
            subTable = subTable[condition];
            return subTable;
        }

        /* lookup tworzony na podstawie własnego View */
        public object GetListKontrah3()
        {
            var view = Session.GetCRM().Kontrahenci.WgNazwy.CreateView();
            // ew. dodatkowe warunki filtrujące
            return view;
        }

        /* lookup tworzony na podstawie LookupInfo, ze wskazaniem dedykowanego ViewInfo,
           a tym samym osobnego lookupform.xml-a */
        public object GetListKontrah4()
        {
            var view = Session.GetCRM().Kontrahenci.WgKodu.CreateView();
            var lookupInfo = new LookupInfo.TableItem(view);
            lookupInfo.ViewInfoType = typeof(KontrahenciViewInfo2); // ViewInfo -> ResourceName -
> Maszyna2 -> Maszyna2.looupform.xml
            /* zastosowanie LookupInfo() daje dodatkowe możliwości */
            lookupInfo.ComboBox = true;
            // ...
            return lookupInfo;
        }

        /* properties-y wykorzystywane w pageform.xml jako EditValue="{Property}" */
        public Kontrahent Kontrah0 { get; set; } // { get; set; } konieczne
        public Kontrahent Kontrah1 { get; set; }
        public Kontrahent Kontrah2 { get; set; }
        public Kontrahent Kontrah3 { get; set; }
        public Kontrahent Kontrah4 { get; set; }
    }
}
```

Kontrahenci.ViewInfo.cs

```
using System;
using System.Collections.Generic;
```

```
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Soneta.Business;
using Soneta.Business.UI;
using PrzykladLookup.UI;

namespace PrzykladLookup.UI
{
    public class KontrahenciViewInfo : ViewInfo
    // View wiążemy z odpowiednią definicją viewform.xml poprzez property ResourceName

    {
        public KontrahenciViewInfo()
        {
            ResourceName = "Kontrahenci";
        }
    }
    // View wiążemy z odpowiednią definicją viewform.xml poprzez property ResourceName
    public class KontrahenciViewInfo2 : KontrahenciViewInfo
    {
        public KontrahenciViewInfo2()
        {
            ResourceName = "Kontrahent2";
        }

    }
}
```

PrzykladowyLookupViewInfo.cs

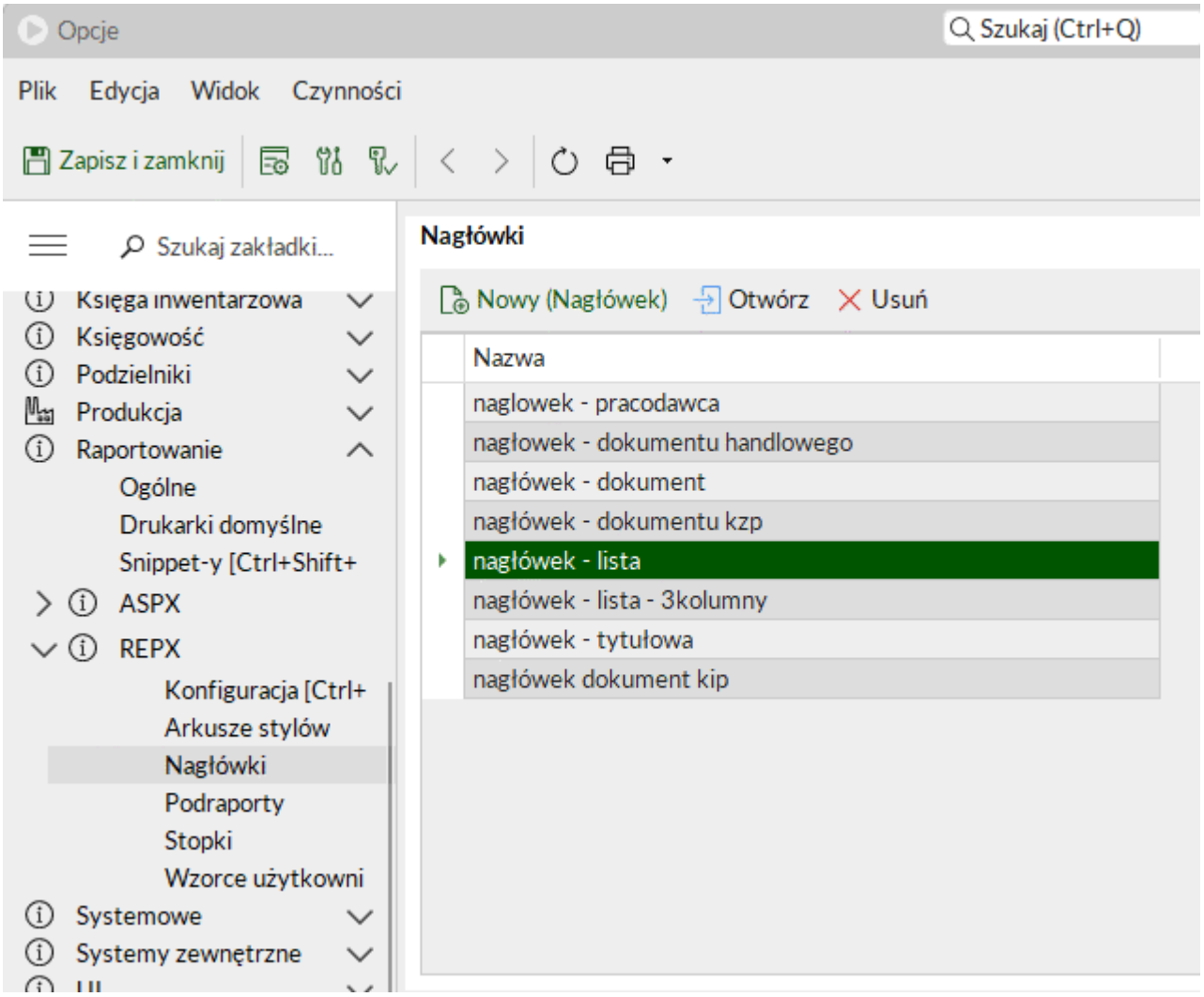
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

using Soneta.Business;
using Soneta.Business.UI;
using PrzykladLookup.UI;

using Soneta.CRM;


[assembly: FolderView("PrzykładowyLookup",
    Priority = 1000,
    Description = "Opis"
```

```
)]

[assembly: FolderView("PrzykładowyLookup/Lookup",
    Priority = 1000,
    Description = "Opis",
    //TableName = "Kontrahenci",
    ViewType = typeof(PrzykladowyLookupViewInfo),
    TableName ="Kontrahent"
)]

namespace PrzykladLookup.UI
{
    class PrzykladowyLookupViewInfo : ViewInfo
    {

    }
}
```

Zapis zmian w konstrukcji podraportów repx

W raportach repx oprócz głównej sekcji raportu można wyróżnić tzw. podraporty takie jak nagłówek i stopka. To składowe elementy raportu, które można edytować tak samo jak raport główny. Podgląd listy podraportów możliwy jest z poziomu menu:

Narzędzia/Opcje/Raportowanie/REPX

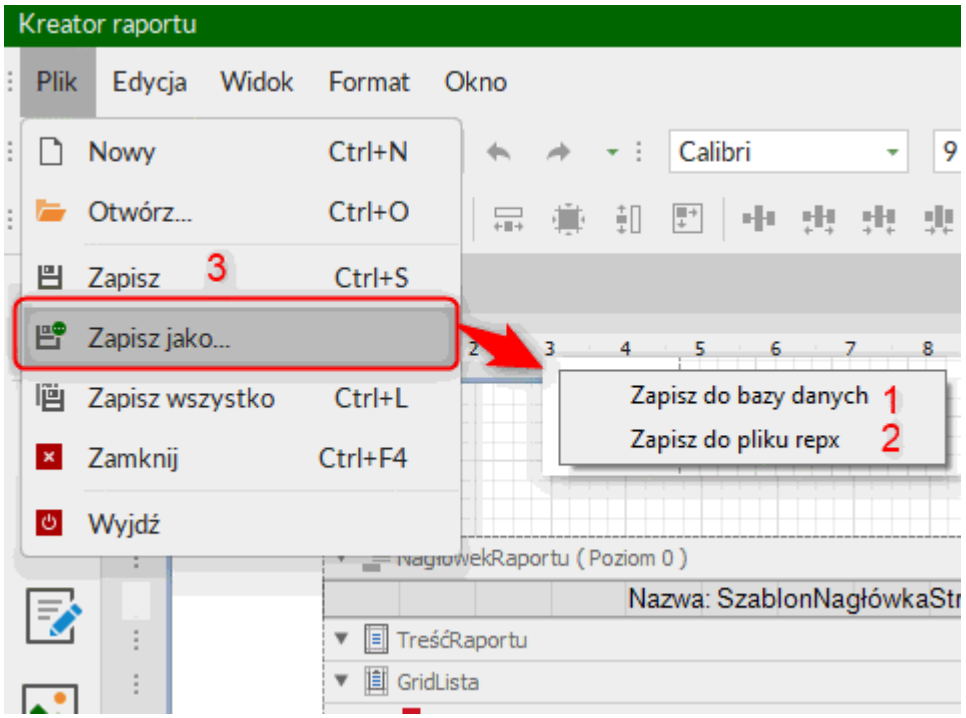


Edycja podraportu wykonywana jest przy pomocy kreatora repx tak samo jak w przypadku raportu głównego. Kreator repx uruchamia się poprzez dwuklik na wybranym podraporcie. Wraz z instalacją systemu enova365 do bazy danych dodawane są podraporty systemowe np. 'nagłówek – lista'. W przypadku raportów systemowych (np. raport repx Dokumenty sprzedaży) gdy operator chce edytować budowę raportu, następuje utworzenie kopii raportu systemowego i przeniesienie jej do tzw. wzorców użytkownika.

Operator dokonując modyfikacji raportu systemowego działa de facto na jego kopii, a dokonane zmiany przypisywane są do wybranego kontekstu zapisu link. Nie ma możliwości by operator dokonał zmian w raporcie w taki sposób by oryginalna postać raportu nie została zachowana w bazie danych.

W przypadku edycji podraportów system enova365 nie wymusza by operator działał na kopii podraportu. Przy otwieraniu kreatora repx podraportu operator nie jest zobligowany do

skopiowania podraportu do wzorców użytkownika. Z poziomu kreatora repx dostępne są różne opcje zapisu zmian dokonanych w podraporcie:



1. Zapisz do bazy danych - skutkuje utworzeniem podraportu we wzorcach użytkownika. Oryginalna postać podraportu nie zostaje utracona. Powstały wzorzec użytkownika widnieje w dwóch miejscach:

- Narzędzia/Opcje/Raportowanie/REPX/Wzorce użytkownika.

Akcja dwukliku uruchamia kreator repx

- Narzędzia/Opcje/Systemowe/Pliki/REPX/Wzorce użytkownika.

Akcja dwukliku otwiera plik tekstowy z kodem c#, w którym zapisana jest konkstrukcja raportu w sensie kontrolek – przenosząc to na porównanie do kreatora repx jest to zapis kontrolek widocznych w zakładce kreatora 'Projekt'.

W pliku nie ma kodu metod związanych z raportem, które z poziomu kreatora repx prezentowane są w zakładce 'Skrypty'.

2. Zapisz do pliku repx - skutkuje utworzeniem pliku na dysku.

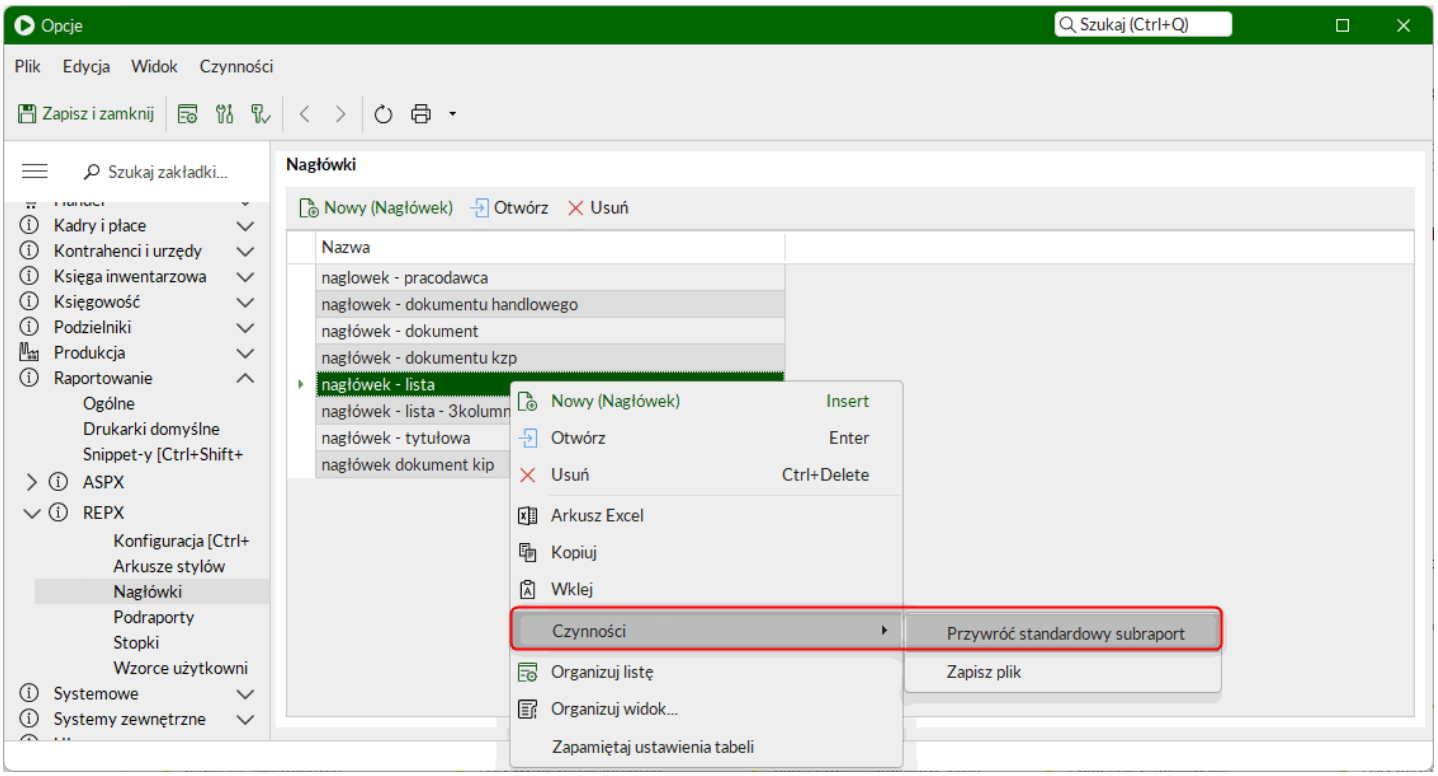
W takim pliku zapisany jest tylko kod 'projektu' podraportu tak jak w przypadku opcji 1b lecz do wzorców użytkownika nie zostaje dodany wpis.

3. Zapisz – skutkuje nadpisaniem konstrukcji podraportu. We wzorcach użytkownika nie powstaje nowy wpis. Może to prowadzić to do stanu, w którym nie ma jak przywrócić pierwotnej konstrukcji podraportu.

Dopiero zapis zmian przez tą opcję skutkuję tym, że mechanizm budowania pełnego raportu konstruje nagłówek, w którym są zmiany wprowadzone przez operatora.

Przed zapisem zmian w podraportach systemowych zaleca się utworzenie backupu z pierwotną konstrukcją podraportu poprzez zapis zmian wykonany przez opcje nr 1 lub 2.

Przywracanie standardowych subraportów



Czynność przywraca budowę subraportu do postaci w jakiej subraport jest dostarczany z daną wersją systemu enova365.

Przed użyciem tej funkcjonalności zaleca się zapisanie backupu jeśli subraport lub jego skrypt był modyfikowany (sposób zapisu backupu opisany w pierwszej części artykułu).

Pieczątki oddziałów w nagłówkach raportów repx przy trybie wielooddziałowości#

Przy trybie wieloodziałowości powstała potrzeba by w nagłówku raportu repx zamieszczać pieczątkę oddziału, z którymi związany jest obiekt źródłowy dla danego raportu.W scenariuszu gdy raport generowany jest z poziomu listy, na której zaznaczone są rekordy powiązane z różnymi oddziałami, oczekiwanym efektem są nagłówki z różnymi pieczątkami. Za określenie oddziału, na podstawie którego definiowana jest pieczątka odpowiada algorytm zdefiniowany w funkcji `GetDepartmentForStamp` klasy `DxReportService`.Parametrem wejściowym tej funkcji jest obiekt, który jest źródłem danych dla generowanego raportu.

Algorytm sprawdza kolejno czy typ obiektu implementuje następujące interfejsy:

1. `IOddziałNagłówkaDokumentuInfo`
2. `IDokumentKsiegowalny`
3. `IElementStrukturyFirmy`
4. `IOddzialProvider`

Jeśli obiekt, który jest źródłem danych implementuje, któryś z tych interfejsów to znaczy, że posiada właściwość **Oddział** a jej wartość jest rezultatem funkcji `GetDepartmentForStamp`.

Jeśli powyższy warunek nie jest spełniony to rezultatem funkcji jest oddział wynikający z ustawionego filtra na widoku.

Jeśli nie ma ustawionego filtra to wynikiem jest oddział powiązany z uprawnieniem zalogowanego operatora (entitle).

Istnieją klasy raportów repx (repx.cs), które posiadają klasy wewnętrzne i to właśnie instancje tych klasy wewnętrznych są źródłem raportów.

Np. raport repx Cesja

Klasa CesjaXtr.repx.cs posiada klasę wewnętrzną **Odcinek**

Robiąc debug generowania raportu można sprawdzić, że to właśnie obiekt typu **Odcinek** jest źródłem raportu.

Takie sprawdzenie umożliwia wywołanie funkcji:

```
reportContext.Report.RootReport.GetCurrentRow();
```

gdzie Report to instancja generowanego raportu typu XtraReport (np. CesjaXtr)

Aby opisany algorytm `GetDepartmentForStamp` określił oddział na podstawie obiektu typu **Odcinek** to ten typ musi implementować interfejs narzucający właściwość **Oddział**.

Jeśli taki warunek nie zajdzie to wg algorytmu rezultatem będzie oddział wynikający z filtra na widoku lub uprawnienia zalogowanego operatora.

Soneta sp. z o.o.
ul. Wadowicka 8A, 30-415 Kraków
+48 12 349 28 00
kontakt@enova.pl